

Généricité

Généricité

- La **généricité** est un mécanisme qui permet d'écrire des **classes**, **interfaces** et **méthodes** paramétrées par des **types**. Autrement dit, On peut créer du code **réutilisable**, **sûr** sans préciser à l'avance **le type exact** des objets manipulés.

- **Type paramétré:**

Les conventions de dénomination de types paramétrés sont importantes pour apprendre la générique. En Java, les types paramétrés courants sont les suivants:

T – Type ;

E – Element;

K – Key;

N – Number;

V – Value

Syntaxe Générale:

Exemple 1 : Classe avec un seul type générique

```
public class NomClasse<T> {  
  
    T valeur;  
  
    public T getValeur() { return valeur; }  
}
```

Exemple 2: Classe avec deux types génériques

```
public class NomClasse<K, V> {  
    private  
    K cle;  
    private  
    V valeur;  
  
    public  
    NomClasse(K cle, V valeur) {  
        this.cle = cle;  
        this.valeur = valeur;  
    }  
    // getters...  
}
```

Exemple1: Classe Générique:

```
public class Boite<T> {  
    private T contenu;  
  
    public void set(T contenu) {  
        this.contenu = contenu;  
    }  
    public T get() {  
        return contenu;  
    }  
}
```

Utilisation:

```
public class Test {  
    public static void main (String []arguments) {  
        Boite<Integer> b1=new Boite<Integer>();  
        b1.set(new Integer(5));  
        System.out.println(' la valeur du contenu est:'+b1.get());  
    }  
}
```

Exemple 2: des piles de n'importe quoi

```
class Pile<E> {  
    private int nb; // Nombre d'elements utilisés  
    private E[] tab; // Attention : pas de creation des  
tableaux generiques ==> les passer dans le  
constructeur.  
    t=new E[2]; est interdit  
    public Pile(E[] tableau) { tab = tableau; nb=0; };  
    public void empiler(E e) { if (nb < tab.length)  
tab[nb++] = e; }  
    public E depiler() {  
    return (nb>0 ?tab[--nb]:null); }  
    public void afficher() { for (int i=0; i<nb;i++)  
System.out.println(tab[i]); }  
}
```

Test: pile d'entiers et pile de point

```
public class Test {  
    // au moment de l'instanciation on précise le type de données à gérer  
    public static void main (String []arguments) {  
        Pile<Integer> p1= new Pile<Integer>(new Integer[10]);  
        p1.empiler(8); p1.empiler(10); p1.afficher();  
  
        Pile<Point> p2 = new Pile<Point>(new Point [8]);  
        Point a = new Point(8,9); p2.empiler(a);  
        p2.empiler(new Point()); p2.empiler(new Point(1,2));  
        p2.afficher();  
    }  
}
```

Interfaces génériques Collection et List

Exemple 3 :

```
public class test {  
    public static void main (String []arg) {  
        Collection<Forme> liste; // Collection de Forme  
        liste = new ArrayList<>(); //tableau de Forme  
        liste.add(new Cercle(1,2,3));  
        liste.add(new Cercle(4,5,6));  
        liste.add(new Rectangle(9,8,7,6));  
        System.out.println("Affichage par un foreach");  
        for ( Forme a : liste) System.out.println(a);  
        System.out.println("Affichage de la surface par  
        iterateur");  
        for (Iterator<Forme> p=liste.iterator();  
        p.hasNext(); ) {  
            Forme a= p.next(); // Pas de conversion utile  
            System.out.println("surface de = "+a.surface());  
        }  
    }  
}
```

Généricité et bornes

Syntaxe des bornes multiples : `<T extends Classe & Interface1 & Interface2>`

Règles importantes : Une seule classe (en premier), puis une ou plusieurs interfaces

Exemple 1: Classe générique avec contrainte forte

```
class Boite<T extends Number & Comparable<T>&Serializable> {  
    private T valeur;  
  
    public Boite(T valeur) {  
        this.valeur = valeur;  
    }  
    public T getValeur() {  
        return valeur;  
    }  
    public boolean estPlusGrandQue(T autre) {  
        return valeur.compareTo(autre) > 0;  
    }  
}
```


Exemple 2 : Généricité et bornes (extends)

On peut restreindre un type générique avec extends pour exiger qu'il hérite d'une classe ou implémente une interface.

```
// Accepte uniquement les nombres (Integer, Double, Float...)
```

```
class Calculatrice<T extends Number> {  
    3 usages  
    private T nombre;  
  
    2 usages  
    public Calculatrice(T nombre) {  
        this.nombre = nombre;  
    }  
  
    2 usages  
    public double carre() {  
        return nombre.doubleValue() * nombre.doubleValue();  
    }  
}
```

Généricité

Exécution

```
class TestBorneGenerics {  
    public static void main(String[] args) {  
        Calculatrice<Integer> c1 = new Calculatrice<>( nombre: 5);  
        System.out.println(c1.carre());  
  
        Calculatrice<Double> c2 = new Calculatrice<>( nombre: 3.5);  
        System.out.println(c2.carre());  
        Calculatrice<Float> c3 = new Calculatrice<>( nombre: 7.05f);  
        System.out.println(c3.carre());  
    }  
}
```

Résultat:

```
25.0  
12.25  
49.70250268936161
```

```
Process finished with exit code 0
```

Méthode Générique

Méthode générique

Les méthodes génériques sont celles qui sont écrites avec **une seule déclaration** et qui peuvent être appelées avec **des arguments de différents types**.

- En fonction des types d'arguments passés à la méthode générique, le compilateur gère chaque appel de méthode de manière appropriée.
- **Voici les règles pour définir les méthodes génériques:**
 - ❖ Les méthodes génériques ont **un type paramétré** avant le type de retour de la déclaration de méthode
 - ❖ Les méthodes génériques peuvent **avoir différents types paramétrés séparés par des virgules** dans la signature de méthode.
 - ❖ Le corps d'une méthode générique est comme une méthode normale.

Méthode Générique

Syntaxe méthode normale:

```
public <T> nomDeLaMethode(T parametre) {  
    // corps de la méthode  
    return parametre;  
}
```

Ou

```
public <T> void nomDeLaMethode(T parametre) {  
    // corps de la méthode  
}
```

Syntaxe méthode staic:

```
public static <T> T nomDeLaMethode(T parametre) {  
    // corps de la méthode  
    return parametre;  
}
```

Méthode Générique

Exemple 1: Afficher un tableau quelconque

```
class Main{
    3 usages
    public static <T> void afficherTableau(T[] tableau) {
        for (T element : tableau) {
            System.out.print(element + " ");
        }
        System.out.println();
    }
    public static void main(String[] args) {
        Integer[] entiers = {1, 2, 3, 4};
        String[] mots = {"Hello", "World"};
        Double[] decimaux = {3.14, 2.71};

        afficherTableau(entiers);
        afficherTableau(mots);
        afficherTableau(decimaux);
    }
}
```

```
C:\Users\jab
1 2 3 4
Hello World
3.14 2.71
```

Wildcards & Bornes (? extends / super)

? (Wildcard non borné)

```
void afficher(  
    List<?> liste)  
{ ... }
```

- Accepte n'importe quel type.
- Utile pour lire sans restriction.

Lecture seule, type inconnu
n'est pas utiliser pour
declarer une classe générique

? extends T (Borne supérieure)

```
double somme(  
    List<? extends  
    Number>  
    nums) { ... }
```

- Accepte Number et ses sous-types (Integer, Double...).

Producteur: lire des données

? super T (Borne inférieure)

```
void ajouter(  
    List<? super  
    Integer>  
    liste) { ... }
```

- Accepte Integer et ses super-types (Number, Object).

Consommateur:

- écrire des données
- Lire des valeurs de type Object(cast obligatoire)

PECS: Producer → extends | Consumer → super

Attention: Les génériques en Java sont invariants : `List<Integer>` n'est PAS un sous-type de `List<Number>`

Méthode Générique : Avec contrainte (bornes)

❖ Méthode générique avec contrainte (bornes) :

Pour restreindre T à des classes comparables par exemple max de deux valeurs

```
public static <T extends Comparable<T>> T max(T a, T b) {  
    return (a.compareTo(b) >= 0) ? a : b;  
}  
  
public static void main(String[] args) {  
    System.out.println(max( a: 10, b: 20));  
    System.out.println(max( a: "Alami", b: "Salhi"));  
    System.out.println(max( a: 3.14, b: 2.71)); }  
}
```

Résultat:

Attention: Ici, T doit implémenter Comparable<T>, donc on peut utiliser compareTo().

20
Salhi
3.14

Méthode Générique : Avec contrainte (bornes)

Exercice:

Écrivez une méthode générique maxElement qui prend un tableau de n'importe quel type comparable (par exemple Integer, Double, String, etc.) et retourne l'élément le plus grand.

Solution:

```
public <T extends Comparable<T>> T maxElement(T[] tableau) {  
    if (tableau == null || tableau.length == 0) return null;  
  
    T max = tableau[0];  
    for (int i = 1; i < tableau.length; i++) {  
        if (tableau[i].compareTo(max) > 0) {  
            max = tableau[i];  
        }  
    }  
    return max;  
}
```


Méthode Générique : Avec contrainte (bornes)

Test:

```
public class Main {  
    static void main() {  
  
        Press Alt Enter with your caret at the highlighted text to see how  
        IntelliJ IDEA suggests fixing it.  
  
        Tableau tableau = new Tableau();  
        Integer entiers[] = {1,2,3,4,5,6,7,8,9,10};  
        String chaines[] = {"alaoui", "elghazi", "mohamed", "yassine"};  
        Float reels[] = {12.5f, 15.6f, 14.8f, 16.9f};  
        System.out.println("max entiers: "+tableau.maxElement(entiers));  
        System.out.println("max mots: "+tableau.maxElement(chaines));  
        System.out.println("max réels: "+tableau.maxElement(reels));  
    }  
}
```

Résultat:

```
max entiers: 10  
max mots: yassine  
max réels: 16.9
```

Méthode Générique : Avec contrainte (bornes)

Exemple max des étudiants:

Si on souhaite afficher
Le max des étudiants,
Il faut faire implémenter
La classe Etudiant de
l'interface Comparable et
préciser le critère de
comparaison dans la méthode
compareTo(Etudiant o).

Ici on comparera les moyennes
des étudiants.

```
public class Etudiant implements Comparable<Etudiant>{  
    2 usages  
    int cne;  
    4 usages  
    String nom;  
    10 usages  
    float moyenne;  
    4 usages  
    public Etudiant(int cne, String nom, float moyenne) {  
        this.cne = cne;  
        this.nom = nom;  
        this.moyenne = moyenne;  
    }  
    @Override  
    public int compareTo(Etudiant o) {  
        if(moyenne>o.moyenne)return 1;  
        else if(moyenne<o.moyenne)return -1;  
        else return 0;  
    }  
    @Override
```

Méthode Générique : Avec contrainte (bornes)

Méthode générique **maxElement(T [] tableau)** inchangeable ,
mais cette fois-ci, on a choisi **static**

```
public static <T extends Comparable<T>> T maxElement(T[] tableau) {  
    if (tableau == null || tableau.length == 0) return null;  
  
    T max = tableau[0];  
    for (int i = 1; i < tableau.length; i++) {  
        if (tableau[i].compareTo(max) > 0) {  
            max = tableau[i];  
        }  
    }  
    return max;  
}
```

Méthode Générique : Avec contrainte (bornes)

Test:

```
public static void main(String[] args) {  
  
    Etudiant [] etudiants= new Etudiant[4];  
  
    etudiants[0]=new Etudiant( cne: 123, nom: "Ghadil", moyenne: 15);  
    etudiants[1]=new Etudiant( cne: 793, nom: "Chakori", moyenne: 11);  
    etudiants[2]=new Etudiant( cne: 183, nom: "Zahiri", moyenne: 7);  
    etudiants[3]= new Etudiant( cne: 183, nom: "Bakkali", moyenne: 13);  
  
    System.out.println("Max Etudiants : " + maxElement(etudiants));  
}
```

Résultat:

```
Max Etudiants : Etudiant{cne=123, nom='Ghadil', moyenne=15.0}
```

Bornes multiples

```
public <T extends Comparable<T> & Serializable>  
void traiter( T obj) {  
    ... }  
}
```

Pattern Repository générique

```
public interface Repository<T, ID> {  
    void sauvegarder(T entity);  
    List<T> trouver(ID id);  
    List<T> tousLesElements();  
}
```

// Implémentation

```
class UtilisateurRepo  
implements Repository<Utilisateur, Long> {  
    ...  
}
```

Méthodes Génériques: Exercices

Exercice 1: Créez une classe générique **Triple<A, B, C>** avec trois attributs privés (un de chaque type), un constructeur, des getters, et une méthode **toString()**.

- Testez-la avec (String, Integer, Boolean) et (Double, String, Character).

Exercice 2: Écrivez une méthode statique générique **inverser(T[] tab)** qui inverse un tableau en place.

- Testez avec Integer[], String[] et Double[].

Exercice 3: Écrivez une méthode générique qui a deux paramètres génériques numériques et qui permet de copier une liste dans une deuxième liste en respectant **PECS**